

Context Switching, Dispatch, and Preemption

Week 4: From scheduling decision to execution on a modern multicore CPU

SDB

Autumn 2026

Intermediate/graduate engineering lecture set

Learning outcomes

By the end of this lecture set, you should be able to:

1. separate scheduling policy, dispatch mechanism, privilege entry, and context switching;
2. trace preemption from interrupt/exception entry to return into a selected thread;
3. identify architecture-dependent and kernel-dependent state switched on a handoff;
4. decompose interrupt, scheduling, dispatch, and wakeup-to-run latency;
5. explain cache, TLB, NUMA, migration, and speculation-related indirect costs;
6. evaluate time-quantum, runnable-load, and memory-pressure trade-offs; and
7. measure switching behavior using counters and scheduler traces without overinterpreting them.

Guiding question

What exact events occur between “thread B should run” and the first useful instruction executed by B ?

Road map

- 1 Mechanism, Policy, and Control Transfer
- 2 What a Context Switch Changes
- 3 Preemption, Interrupts, and Kernel Execution
- 4 Latency Decomposition and Performance
- 5 Multicore Dispatch and System Load
- 6 Implementation and Observability
- 7 Synthesis

Road map

- 1 Mechanism, Policy, and Control Transfer
- 2 What a Context Switch Changes
- 3 Preemption, Interrupts, and Kernel Execution
- 4 Latency Decomposition and Performance
- 5 Multicore Dispatch and System Load
- 6 Implementation and Observability
- 7 Synthesis

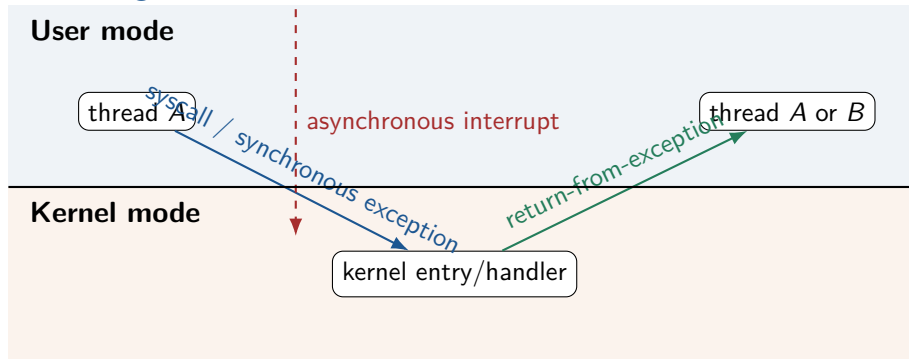
Four concepts that must remain separate

Concept	Question answered	Example
Scheduling policy	which runnable thread should run, where, and for how long?	fair share, priority, EDF
Scheduler mechanism	how are runnable tasks represented and selected?	per-CPU run queues, class call-backs
Dispatch/switch mechanism	how does the CPU begin executing the selected thread?	stack/task/address-space switch
Privilege transfer	how does execution enter/leave kernel mode?	syscall, exception, interrupt, return

Central correction

A mode switch is not necessarily a context switch. A timer interrupt does not necessarily select a different thread. A context switch can occur between kernel threads without returning to user mode immediately.

How the kernel gains control



- Hardware saves a defined entry state and transfers to a registered kernel entry path.
- Kernel code determines whether the current thread can resume, must block, or should be preempted.
- Return restores an execution context and privilege level appropriate to the selected destination.

When is the scheduler considered?

Common scheduling points include:

Voluntary/blocking paths

- waiting for I/O, a lock, or an event;
- sleeping until a timer/deadline;
- explicit yield;
- termination.

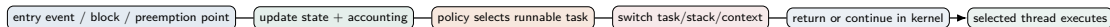
Preemptive/wakeup paths

- quantum or policy accounting expires;
- a higher-priority thread becomes runnable;
- load balancing requests migration;
- returning from interrupt/syscall with reschedule pending.

Kernel execution is not a paradox

Scheduler code runs because an entry path or current kernel path already has CPU control. Kernel threads are schedulable tasks; ordinary scheduler functions are code executed in the current kernel context.

Conceptual scheduling and dispatch path



- If policy reselects the current thread, the expensive task-switch stage may be skipped.
- The selected thread can resume inside a kernel path before later returning to user mode.
- On multicore systems, enqueue/wakeup and eventual execution may occur on different CPUs.

Road map

- 1 Mechanism, Policy, and Control Transfer
- 2 What a Context Switch Changes**
- 3 Preemption, Interrupts, and Kernel Execution
- 4 Latency Decomposition and Performance
- 5 Multicore Dispatch and System Load
- 6 Implementation and Observability
- 7 Synthesis

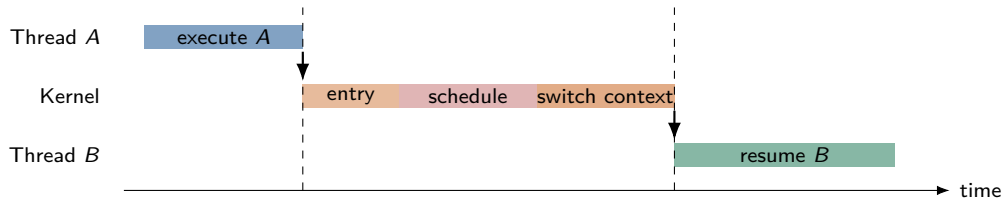
Context is layered, not one flat PCB snapshot

Layer	Potential state	When changed
architectural	PC, SP, flags, general registers	every switched execution stream as ABI requires
extended CPU	FP/vector, matrix, debug, protection keys	eager/lazy/conditional policy
kernel execution	kernel stack, current-task pointer, preemption/lock context	task handoff
memory translation	page-table root, ASID/PCID, protection context	address-space change
scheduler	run-queue links, runtime, class/priority, affinity	scheduling/accounting paths
shared process state	files, mappings, credentials, signals	normally referenced, not copied on every switch
microarchitecture	caches, TLB entries, predictors, prefetch state	affected indirectly; not normally saved/restored

Correction

Open descriptors, PID, and most resource metadata are not loaded into CPU registers during dispatch. They remain kernel objects referenced by the selected task/process.

Low-level handoff: a conceptual timeline



1. Preserve *A*'s resumable state and record why it stopped.
2. Select *B* and update run-queue/accounting state.
3. Change kernel stack/current-task and address-space context if required.
4. Restore *B*'s state and continue at its saved kernel/user return path.

Three handoffs with different costs

Same thread

syscall/interrupt enters kernel and returns to the same thread.

No scheduler task switch required.

Sibling threads

switch between threads sharing an address space.

Stacks/registers change; mappings may remain.

Different processes

thread and address-space context may change.

Translation/security and cache effects can grow.

But do not rank blindly

A migrated sibling thread with a cold working set can cost more than a same-CPU process switch whose translations are tagged and whose cache footprint is small.

Address-space switching, TLBs, and tagged translations

- Page-table context tells the MMU how to translate virtual addresses.
- ASIDs/PCIDs tag translations by address space so entries can coexist in the TLB.
- Switching the page-table root need not flush every translation.
- Mapping changes still require targeted invalidation and cross-CPU TLB shootdowns.
- Kernel/user page-table isolation and security mitigations can add entry/switch work.

Indirect penalty

Even without a full TLB flush, the selected task can suffer capacity/conflict misses and page-table walks because another workload displaced useful entries.

Multicore detail

A mapping update may interrupt other CPUs using that address space so stale translations are invalidated safely.

Extended CPU state: vectors, matrices, and security

- Modern extended register files can be large: floating-point, SIMD/vector, matrix/tile, debug, and control state.
- **Eager switching** saves/restores state predictably but may move unused data.
- **Lazy switching** defers restoration until first use, reducing some work but complicating faults, ownership, and security.
- Kernels may track which features a thread used and save variable-sized state selectively.
- Real-time systems often prefer predictable eager behavior over average-case optimization.

Security history lesson

Lazy state management can create information-leak hazards if ownership and speculation are not handled correctly. Optimization policy must preserve isolation.

Microarchitectural state is not restored like registers

Potentially disrupted

- private/shared cache contents;
- TLB and paging-structure caches;
- branch predictors;
- hardware prefetch streams;
- memory-controller and interconnect queues.

Consequences

- slower initial execution after dispatch;
- interference among co-runners;
- side channels across protection domains;
- high variance despite similar direct switch time.

Working-set principle

The indirect cost depends on the outgoing and incoming working sets, cache topology, CPU placement, and time since the incoming task last ran.

Road map

- 1 Mechanism, Policy, and Control Transfer
- 2 What a Context Switch Changes
- 3 Preemption, Interrupts, and Kernel Execution**
- 4 Latency Decomposition and Performance
- 5 Multicore Dispatch and System Load
- 6 Implementation and Observability
- 7 Synthesis

Preemption requires a safe point

The kernel may temporarily defer preemption while:

- holding a spinlock or manipulating CPU-local invariants;
- executing hard-interrupt context;
- interrupts or preemption are explicitly disabled;
- architecture entry/exit state is incomplete;
- a non-preemptible critical section is active.

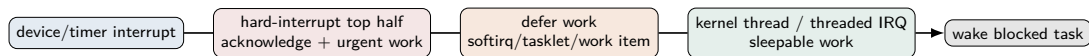
Reschedule request

A timer or wakeup can mark that rescheduling is needed. The actual switch occurs when execution reaches a legal scheduling point.

Latency implication

Long non-preemptible regions, interrupt masking, or high-priority handlers can delay a ready high-priority task.

Interrupt work is split to control latency



- Keep hard-interrupt handling bounded because it delays other interrupt and scheduling work.
- Deferred mechanisms differ in whether they can sleep, their priority, CPU affinity, and concurrency.
- Real-time configurations often thread more interrupt work so it can be prioritized and preempted.

Timer interrupts enable accounting, not automatic fairness

1. A hardware timer event causes a privileged interrupt entry.
2. Kernel timekeeping and scheduler accounting update elapsed runtime/timers.
3. Policy may mark the current task for preemption or wake timed sleepers.
4. At a valid scheduling point, the scheduler can choose a runnable task.

No guarantee from the timer alone

Fairness and responsiveness depend on scheduling policy, priority configuration, runnable load, non-preemptible latency, and resource contention. Timer ticks merely provide one control mechanism.

Tickless kernels

Modern kernels can program one-shot timers and suppress periodic ticks when possible. Preemption does not require a fixed global tick interval.

Kernel preemption models

Model	Behavior	Trade-off
non-preemptive kernel	switch mainly on re- turn/block/explicit points	simple invariants; potentially long latency
voluntary preemption	explicit checks in long paths	improved latency; coverage depends on check placement
preemptible kernel	most process-context kernel code preemptible outside protected regions	lower latency; more synchronization complexity
real-time/preempt-RT style	thread interrupt/lock paths where feasible; priority-aware locking	more deterministic latency; overhead and integration constraints

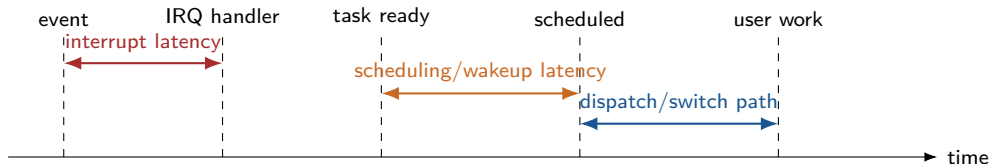
Design goal

Bound the longest interval during which a higher-priority ready task cannot execute, not merely the average context-switch instruction count.

Road map

- 1 Mechanism, Policy, and Control Transfer
- 2 What a Context Switch Changes
- 3 Preemption, Interrupts, and Kernel Execution
- 4 Latency Decomposition and Performance**
- 5 Multicore Dispatch and System Load
- 6 Implementation and Observability
- 7 Synthesis

Latency terms: define the interval before measuring



- **Interrupt latency**: event to beginning of its handler, under a stated definition.
- **Wakeup-to-run latency**: runnable transition to execution on a CPU.
- **Dispatch latency**: scheduling decision/dispatch point to selected execution; definitions vary.
- **Response time**: workload-level release/request to first response or completion, depending on field.

Direct and indirect switch cost

A first-order decomposition is

$$T_{\text{handoff}} = T_{\text{entry}} + T_{\text{scheduler}} + T_{\text{switch}} + T_{\text{return}} + T_{\text{warmup}}.$$

Direct/measured in kernel path

- entry and accounting;
- run-queue selection/locking;
- architectural state handoff;
- return/mitigation work.

Indirect/workload-visible

- cache/TLB/predictor warmup;
- migration and NUMA access;
- queueing before dispatch;
- interference from co-runners.

The terms overlap depending on instrumentation; the equation is a reasoning framework, not a hardware identity.

Time quantum: overhead versus response

Suppose useful execution lasts quantum q and a handoff costs s . Under a simple saturated round-robin model:

$$\eta_{\text{direct}} \approx \frac{q}{q + s}, \quad f_{\text{switch}} \approx \frac{1}{q + s}.$$

Smaller q can:

- reduce worst-case turn waiting;
- improve interactive sharing;
- increase switch/cache overhead;
- fragment CPU bursts.

Larger q can:

- improve locality/throughput;
- approach FCFS behavior;
- increase response delay under load;
- let latency-sensitive work wait longer.

Example: direct overhead is nonlinear in the quantum

Let $s = 6\ \mu\text{s}$.

q	overhead $s/(q + s)$	useful efficiency $q/(q + s)$
20 ms	0.030%	99.970%
1 ms	0.596%	99.404%
100 μs	5.66%	94.34%

Interpret carefully

These percentages cover only the assumed direct $6\ \mu\text{s}$ cost. Cache/TLB disruption, migration, queueing, and shortened useful bursts may dominate.

Tail latency matters more than the average

Sources of long tails:

- interrupt storms or long handlers;
- lock contention and priority inversion;
- CPU frequency/idle-state exit;
- NUMA migration and cache misses;
- page faults, reclaim, and write-back;
- virtualization steal time.

Report distributions

Measure median, high percentiles, maximum under a defined test window, and outliers with causal traces. A low mean does not establish a real-time bound.

Coordinated omission

A load generator that pauses during stalls can underreport exactly the latency experienced by a fixed-rate workload.

Real-time focus: bounded interference

A high-priority task's release-to-run latency can include:

$$L \leq B_{\text{irq-off}} + B_{\text{nonpreempt}} + B_{\text{lock}} + I_{\text{higher-priority}} + C_{\text{dispatch}},$$

under an explicit analysis model.

- B terms bound interrupt masking, non-preemptible regions, and resource blocking.
- I captures interference from work allowed to outrank the task.
- C_{dispatch} includes the relevant scheduler/switch path.

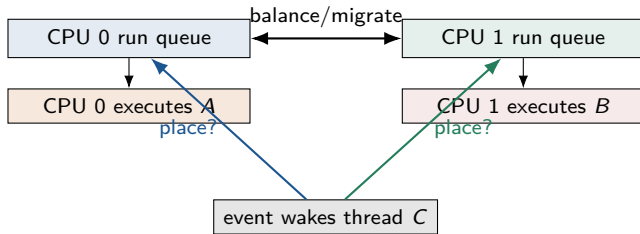
Not a complete theorem

Hardware contention, DMA, cache effects, firmware/system-management interrupts, release jitter, and multicore interference must be modeled or controlled for a defensible bound.

Road map

- 1 Mechanism, Policy, and Control Transfer
- 2 What a Context Switch Changes
- 3 Preemption, Interrupts, and Kernel Execution
- 4 Latency Decomposition and Performance
- 5 Multicore Dispatch and System Load**
- 6 Implementation and Observability
- 7 Synthesis

Per-CPU run queues and wakeup placement



- Per-CPU queues reduce global locking but create imbalance and placement decisions.
- Wakeup policy considers idle CPUs, previous CPU/cache affinity, NUMA, priority, energy, and capacity.
- Cross-CPU wakeups may require inter-processor interrupts; migration can disturb locality.

Heterogeneous CPUs and capacity-aware dispatch

Modern systems may combine cores with different:

- peak performance and energy efficiency;
- vector/matrix capability or frequency range;
- cache topology and memory proximity;
- thermal headroom and current load.

Policy problem

Place latency-critical or compute-heavy tasks on sufficient-capacity CPUs while preserving energy efficiency and avoiding migrations.

Observation

“CPU utilization” does not capture whether work ran on an appropriate-capacity core or was throttled by power/thermal policy.

Degree of multiprogramming: modern interpretation

The historical term means how many jobs/processes are admitted into active competition. On modern systems, separate:

- **runnable load**: tasks competing for CPU capacity;
- **memory working sets**: active pages competing for RAM/cache/TLB capacity;
- **blocked concurrency**: tasks waiting on I/O, locks, or remote services;
- **resource-control admission**: cgroups, quotas, pools, and service limits;
- **in-flight requests**: application concurrency and queue depth.

Correction

More resident processes do not directly imply more context switches. Switching frequency depends on runnable tasks, blocking/wakeup behavior, quantum/policy, CPU count, and synchronization.

Queueing explains why excess concurrency hurts latency

Little's law for a stable system relates average in-flight work N , throughput λ , and average time W :

$$N = \lambda W.$$

Increasing concurrency can initially:

- hide I/O latency;
- keep CPUs/devices utilized;
- raise throughput toward capacity.

Beyond bottleneck capacity it often:

- grows queues and tail latency;
- increases cache and lock contention;
- amplifies memory pressure;
- adds work without throughput gain.

Thrashing is a working-set failure, not switch overhead

Thrashing occurs when active memory demand exceeds available capacity so page faults/reclaim dominate useful work.

- major faults can wait on storage;
- reclaim/write-back consumes CPU and I/O;
- tasks repeatedly evict each other's useful pages;
- adding work can reduce throughput.

Context switches are secondary

Faults block and wake tasks, so switching may rise, but the root problem is insufficient memory for active working sets and poor admission/control.

Mitigation

Reduce concurrency, enforce memory limits, improve locality, add memory, tune reclaim, or redesign the work-load/data path.

Road map

- 1 Mechanism, Policy, and Control Transfer
- 2 What a Context Switch Changes
- 3 Preemption, Interrupts, and Kernel Execution
- 4 Latency Decomposition and Performance
- 5 Multicore Dispatch and System Load
- 6 Implementation and Observability**
- 7 Synthesis

Linux: useful implementation landmarks

- schedulable entities are represented by per-thread task records with shared process objects;
- scheduler classes implement different policies over per-CPU run queues;
- a low-level architecture-specific switch changes task/stack and required machine state;
- wakeups, balancing, affinity, cgroups, and NUMA influence placement;
- preemption configuration changes where kernel execution can be displaced.

Avoid stale slogans

Linux is not accurately summarized as “CFS uses a red-black tree” for every task and kernel version. Multiple scheduler classes and evolving fair-scheduler implementations coexist.

Stable lesson

Separate policy/class logic from the architecture-specific handoff and from shared task/process state.

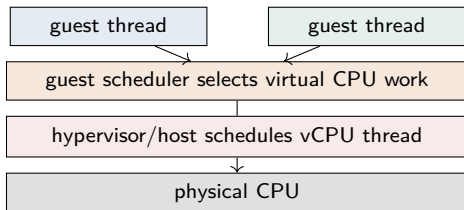
Windows: analogous concepts, different organization

- Threads are dispatchable execution entities; processes supply address-space and resource context.
- Priority-driven preemptive scheduling uses per-processor and system coordination structures.
- Deferred Procedure Calls and interrupt request levels defer/order kernel work under Windows-specific rules.
- Context switching and trap handling include architecture-specific state plus kernel dispatcher bookkeeping.
- Processor groups, affinity, heterogeneous scheduling, and power management affect placement on modern systems.

Comparison discipline

Compare observable semantics and concrete paths rather than forcing Linux names such as run queue or softirq onto Windows internals.

Virtualization adds another scheduling layer



- A guest can schedule a thread onto a vCPU that the host is not currently running.
- Oversubscription, vCPU preemption, interrupt virtualization, and lock-holder preemption add latency variance.
- Guest CPU utilization can hide host “steal” time or placement problems.

Observe counters first, then trace causality

```
1 # Per-thread snapshots and switch rates
2 ps -eLo pid,tid,psr,stat,ni,rtprio,comm
3 pidstat -w -t 1
4
5 # Aggregate counters for a workload
6 perf stat -e context-switches,cpu-migrations,page-faults -- ./workload
7
8 # Scheduler events for causal timing (permissions may be required)
9 perf sched record -- ./workload
10 perf sched timehist
```

Interpretation rule

A high switch count can be healthy asynchronous concurrency or pathological contention. Determine who blocked, who woke it, runnable delay, CPU migration, and the work completed per switch.

Microbenchmark pitfalls

A useful switch benchmark must control:

- same CPU versus migration;
- sibling threads versus processes;
- warm versus cold working sets;
- CPU frequency and idle states;
- competing interrupts/tasks;
- clock source and measurement overhead.

Do not use `sched_yield()` as a universal switch timer

Yield semantics depend on policy and runnable competitors; the caller can be selected again, and the result mixes scheduler policy with switch cost.

Report

Full distribution, topology, affinity, task relationship, kernel configuration, mitigations, and source code.

Off-CPU analysis finds where threads actually wait

- On-CPU profiles show instructions consuming CPU time.
- Off-CPU traces attribute blocked intervals to locks, I/O, timers, paging, or scheduler delay.
- Wakeup-to-run traces separate “event completed” from “thread received a CPU.”
- CPU-migration and cache-counter correlation can expose locality losses.
- Pressure-stall information distinguishes CPU, memory, and I/O contention at system/cgroup scope.

Diagnostic sequence

Identify the latency interval, find its queue, determine the wakeup event, measure runnable delay, and then inspect dispatch/warmup cost.

Road map

- 1 Mechanism, Policy, and Control Transfer
- 2 What a Context Switch Changes
- 3 Preemption, Interrupts, and Kernel Execution
- 4 Latency Decomposition and Performance
- 5 Multicore Dispatch and System Load
- 6 Implementation and Observability
- 7 **Synthesis**

End-to-end preemption example



- Interrupt arrival, task wakeup, scheduler decision, low-level handoff, and useful execution are distinct timestamps.
- If preemption is temporarily disabled, *B* is runnable but cannot yet execute.
- Tail latency may be dominated by handler work, locks, CPU placement, or cache warmup rather than register save/restore.

Misconceptions corrected

Misconception	⇒ Precise statement
A context switch always follows a syscall. The dispatcher loads the whole PCB.	⇒ Same-thread kernel entry/return needs no task switch. ⇒ It changes required execution context; shared resources remain referenced objects.
Every timer interrupt switches tasks.	⇒ It updates time/accounting; policy may retain the current task.
All registers are saved on every switch.	⇒ State depends on architecture, ABI, usage, and eager/lazy policy.
More processes directly cause more switching.	⇒ Runnable/wakeup behavior, policy, CPU count, and quantum determine switches.
Thrashing is excessive context switching.	⇒ It is failure to retain active memory working sets; switching is secondary.
Low average switch time means real-time safety.	⇒ Bounds require worst-case interference and non-preemptible analysis.
The kernel has unrestricted direct access to all physical memory.	⇒ Kernel access follows architecture mappings, permissions, isolation, and device/IOMMU mechanisms.

Key takeaways

- Scheduling chooses; dispatch and context-switch mechanisms realize the decision.
- Kernel entry, mode switch, scheduler invocation, task switch, and user return are distinct events.
- Direct switch instructions are only part of cost; working-set disruption and queueing dominate many workloads.
- Preemption happens only at legal points; interrupt masking, locks, and handlers determine latency tails.
- Multicore dispatch includes wakeup placement, balancing, affinity, NUMA, capacity, and migration.
- Concurrency should saturate useful resources without creating unbounded queues or memory thrashing.
- Measurement must define endpoints and combine counters with causal traces.

Laboratory: decompose wakeup-to-run latency

1. Create a producer that periodically wakes a blocked worker through a pipe/eventfd.
2. Pin both threads to one CPU, then to separate CPUs; record scheduling events.
3. Measure event timestamp, wakeup timestamp, dispatch timestamp, and first worker instruction.
4. Add CPU competition, memory pressure, and a migrating configuration separately.
5. Report distributions and attribute the new delay to queueing, migration, faults, or warmup.

Method

Change one factor at a time and include tracing overhead/control runs. Do not report one minimum value as system latency.

Next week: scheduling algorithms

- FCFS, SJF/SRTF, round robin, priority, and feedback queues;
- response, waiting, turnaround, throughput, fairness, and tail latency;
- starvation, aging, priority inversion, and admission control;
- schedule construction and quantitative metric comparison;
- linking algorithm assumptions to modern multicore implementations.

Preparation

For three jobs with different arrivals and CPU bursts, draw FCFS, SRTF, and round-robin schedules. State every tie-breaking assumption.

Road map

8 Appendix

Knowledge check: mechanism

1. Can a mode switch occur without a context switch? Give two examples.
2. Can a context switch occur without an immediate return to user mode?
3. Why might sibling-thread switching preserve the address-space context?
4. What prevents a runnable high-priority task from preempting immediately?
5. Why can tagged TLBs reduce but not eliminate translation warmup cost?
6. Which state is normally referenced rather than copied during dispatch?

Knowledge check: performance

1. Distinguish interrupt latency, wakeup-to-run latency, and switch-path time.
2. Why can same-process switching be slower than a process switch?
3. What does a context-switch counter fail to reveal?
4. Why does a smaller quantum eventually reduce throughput?
5. How can CPU migration increase latency without changing direct switch code?
6. Why is a low mean insufficient for a hard real-time claim?

Exercise: quantum sensitivity

A saturated one-CPU system runs $n = 8$ equal tasks under ideal round robin. Direct handoff cost is $s = 5 \mu\text{s}$.

1. Compute direct efficiency for $q = 10 \text{ ms}$, 1 ms , and $100 \mu\text{s}$.
2. Estimate the maximum time before a just-missed task receives service under the ideal model.
3. Add a $20 \mu\text{s}$ average cache-warmup penalty and recompute useful efficiency.
4. Explain why averages cannot be added this way for a worst-case bound.

Exercise: reconstruct the latency path

A high-priority thread becomes runnable at $t = 10\ \mu\text{s}$, but executes at $t = 85\ \mu\text{s}$. Trace data shows:

- interrupt handler completes at $30\ \mu\text{s}$;
 - preemption is disabled until $60\ \mu\text{s}$;
 - scheduler and handoff finish at $72\ \mu\text{s}$;
 - first cache-miss-heavy useful operation finishes at $85\ \mu\text{s}$.
1. Attribute intervals to handler, non-preemptible, switch, and warmup components.
 2. Which component should each optimization target?
 3. Which timestamps would a simple switch counter omit?

Exercise: concurrency and queueing

A service completes 4,000 requests/s at saturation. At 20 in-flight requests, mean latency is 5 ms.

1. Verify consistency with Little's law.
2. If concurrency rises to 100 with unchanged throughput, what mean latency follows in steady state?
3. List reasons throughput may instead fall: cache pressure, locks, context switching, memory reclaim, downstream saturation.
4. Propose an admission-control signal and a safe control objective.

Optional tracing commands

```
1 # Per-thread switches and migrations once per second
2 pidstat -w -t 1
3
4 # Scheduler timeline for one command
5 perf sched record -- ./workload
6 perf sched timehist
7
8 # Counters (names/support vary by platform)
9 perf stat -e task-clock,context-switches,cpu-migrations,page-faults \
10 -- ./workload
```

Run only with appropriate permissions. Trace buffers, symbol access, and event availability vary by kernel and security policy.

Further study

- architecture entry/switch assembly and extended-state management;
- preemptible kernels, PREEMPT_RT, threaded interrupts, priority inheritance;
- scheduler tracepoints, off-CPU flame graphs, eBPF latency analysis;
- TLB shootdowns, ASIDs/PCIDs, cache partitioning, NUMA balancing;
- heterogeneous scheduling, energy-aware placement, thermal control;
- vCPU scheduling, steal time, nested virtualization, real-time guests.

Questions?